অ্যানালাইসিস, মডেলিং, ডিজাইন ও আর্কিটেকচার

প্রয়োজনীয়তা · DFD · ER মডেলিং · ডিজাইন নীতি · আর্কিটেকচার — ১২ পাতায়



↑ পুরো টপিক এক পাতায়: লাল = বেশি আসে, সবুজ = প্রয়োগ

সিলেবাসের যে অংশ

"Analysis Concepts and principles: requirement analysis, **Analysis modeling**, **data modeling**. **Design concepts and principles**, **Architectural design**, User Interface design"। ইউজ কেস, ইউনিফাইড প্রসেস ও পূর্ণ UML আলোচনা আছে ইউজ কেস / OO ডিজাইন নোটে।

হাই-ইল্ড তালিকা

① অ্যানালাইসিস = কী, ডিজাইন = কীভাবে
② কনটেক্সট ডায়াগ্রাম = একটি প্রসেস
③ DFD ব্যালান্সিং নিয়ম
④ ফাংশনাল কোহিশন সেরা
⑤ ডেটা কাপলিং সেরা, কনটেন্ট সবচেয়ে খারাপ
⑥ M:N → নতুন রিলেশন, ২টি FK

সংকেত: **হলুদ** = মূল শব্দ · **লাল বৃত্ত** = মুখস্থ রাখার তথ্য · **বেগুনি বক্স** = মনে রাখার কৌশল · **লাল বক্স** = পরীক্ষার ফাঁদ · **সবুজ** = সমাধানকৃত উদাহরণ

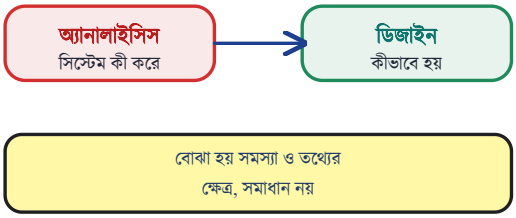
সূচিপত্র

✓ অ্যানালাইসিস ও ছয় নীতি … পৃ. ২
✓ কার্যকরী বনাম অ-কার্যকরী; SRS … পৃ. ২
✓ আহরণ: চারটি পদ্ধতি … পৃ. ৩
✓ সম্ভাব্যতা TELOS ও বৈধকরণ … পৃ. ৩
✓ DFD: উপাদান, কনটেক্সট, ব্যালান্সিং … পৃ. ৪
✓ ডেটা ডিকশনারি + লাইব্রেরি DFD … পৃ. ৫
✓ ER: এন্টিটি, অ্যাট্রিবিউট, সম্পর্ক … পৃ. ৬
✓ লাইব্রেরি ERD → রিলেশনাল স্কিমা … পৃ. ৭
✓ কোহিশন, কাপলিং, ডিজাইন নীতি … পৃ. ৮
✓ ডিজাইনের স্তর, আর্কিটেকচার, UI … পৃ. ৯–১০

১ অ্যানালাইসিস: "কী" আগে, তারপর "কীভাবে"

সংজ্ঞা: **সফটওয়্যার অ্যানালাইসিস** = ডিজাইন ও ইমপ্লিমেন্টেশনের আগে **সফটওয়্যারটি কী করা উচিত তা বোঝা ও নির্দিষ্ট করা**। এটি সমস্যা ও তথ্যের ক্ষেত্র বুঝে, সমাধান বুঝে নয়। অ্যানালাইসিসের উত্তর **কী**, ডিজাইনের উত্তর **কীভাবে**, ইমপ্লিমেন্টেশন হলো করা।

নীতি	এর কী চাই
সঠিকতা	প্রতিটি প্রকৃত প্রয়োজন ধরা পড়ে
ধারাবাহিকতা	দুটি প্রয়োজনীয়তা দ্বন্দ্বী নয়
সম্পূর্ণতা	কার্যকরী + অ-কার্যকরী, দুটোই
সম্ভাব্যতা	সীমার মধ্যেই অর্জনযোগ্য
ট্রেসেবিলিটি	উৎস থেকে টেস্ট পর্যন্ত ট্রেসযোগ্য
বোধগম্যতা	সব পাঠক এক অর্থে পড়েন



দুই ধাপের পার্থক্য এক বাক্যে

২ কার্যকরী বনাম অ-কার্যকরী প্রয়োজনীয়তা

	কার্যকরী (functional)	অ-কার্যকরী (non-functional)
উত্তর দেয়	কোন কোন সেবা ও আচরণ	কতটা ভালো ও কতটা বেশি
উদাহরণ	লগইন, সার্চ, বই ইস্যু	২ সেকেন্ডে উত্তর; ৯৯.৯% আপটাইম; AES-256
কে চায়	শেষ ব্যবহারকারী ও বিশ্লেষক	ব্যবহারকারী **ও** প্রতিষ্ঠান

মনে রাখার কৌশল: তিন স্তর: **ব্যবসায়িক** "ওভারডিউ ফেরত কমানো" → **ব্যবহারকারী** "কর্মীরা ডিউ-ডেট তালিকা দেখেন" → **সিস্টেম** "কোয়েরি ২ সেকেন্ডে ফেরত আসে"।

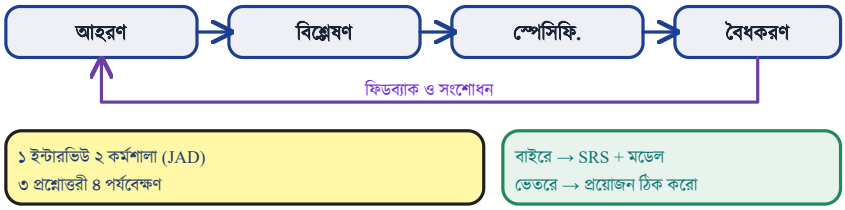
পরীক্ষার ফাঁদ: "সিস্টেম দ্রুত হবে" — এটি প্রয়োজনীয়তা নয়, যাচাইযোগ্যই নয়। প্রতিটি অ-কার্যকরী প্রয়োজনের সঙ্গে **মাপার মানদণ্ড** থাকতে হয়।

৩ স্পেসিফিকেশন: SRS-এ কী থাকে

SRS (IEEE 830 / ISO-IEC 29148): **১ ভূমিকা** — উদ্দেশ্য, পরিধি, সংজ্ঞা, সারসংক্ষেপ, তথ্যসূত্র · **২ সামগ্রিক বর্ণনা** — দৃষ্টিভঙ্গি, ফাংশন, ব্যবহারকারী শ্রেণি, পরিবেশ, সীমাবদ্ধতা, অনুমান · **৩ নির্দিষ্ট প্রয়োজন** — বাইরের ইন্টারফেস, কার্যকরী, কর্মক্ষমতা, নকশা-সীমাবদ্ধতা, সিস্টেম বৈশিষ্ট্য · **৪ সংযোজন** — শব্দতালিকা, সূচক।

৪ প্রয়োজনীয়তা আহরণ: চারটি পদ্ধতি

পদ্ধতি	শক্তি	দুর্বলতা
ইন্টারভিউ	গভীর তথ্য; সঙ্গে সঙ্গে প্রশ্ন করা যায়	ধীর ও ব্যয়বহুল; প্রশ্নকারীর পক্ষপাত
কর্মশালা / **JAD**	দ্রুত; ঐকমত্য তৈরি; মালিকাম্বোধ	দক্ষ সফাস্টকারী দরকার
প্রশ্নোত্তরী	সস্তা; পুনরাবৃত্তিযোগ্য; বহু মানুষ	উত্তর কম; ব্যাখ্যা করা যায় না
পর্যবেক্ষণ	বাস্তবে **যেভাবে করা হয়** তা ধরে	ধীর; লক্ষ্যে থাকলে আচরণ বদলায়



প্রয়োজনীয়তা প্রকৌশল একটি রেখা নয়, বর্তনীয় প্রক্রিয়া

৫ সম্ভাব্যতা যাচাই: TELOS

- **T**echnical — প্রযুক্তিগত: প্রচলিত হার্ডওয়্যার, সফটওয়্যার ও দক্ষতা দিয়ে কি এটি বানানোই যায়?
- **E**conomic — অর্থনৈতিক: সুবিধা কি ব্যয়ের চেয়ে বেশি? (পেব্যাক পিরিয়ড)
- **L**egal — আইনগত: আইন, গোপনীয়তা, লাইসেন্স, চুক্তি, মান
- **O**perational — পরিচালনগত: প্রতিষ্ঠানের কাজের ধরনের সঙ্গে মানাবে?
- **S**chedule — সময়সূচিগত: কত সময়ের মধ্যে ডেলিভারি সম্ভব?

মনে রাখার কৌশল: শব্দ হিসেবে মনে রাখো: **T-E-L-O-S** — গ্রিক *telos* = লক্ষ্য। রিপোর্ট শেষে **গো-নো-গো** সিদ্ধান্ত।

পরীক্ষার ফাঁদ: সম্ভাব্যতা যাচাই হয় SRS লেখার **আগে**, আবার মডেল তৈরির সময় — এটি শেষ ধাপ নয়।

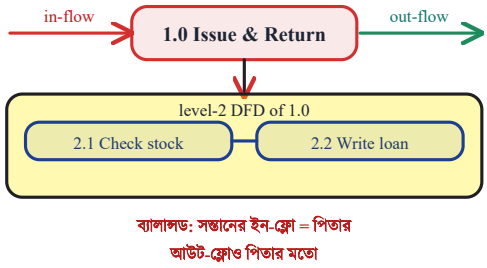
বৈধকরণ (validation): প্রমাণ করা যে মডেল ও স্পেস ঠিক: পুনরীক্ষা ও পরিদর্শন, প্রোটোটাইপ, গ্রহণযোগ্যতার টেস্ট কেস, মডেল ওয়াকথ্রু, ডিসিশন টেবিল। যাচাই করি: সঠিকতা, সম্পূর্ণতা, ধারাবাহিকতা, বাস্তবতা, যাচাইযোগ্যতা।

কাজের উদাহরণ ✍️: একটি হাসপাতাল অনলাইনে অ্যাপয়েন্টমেন্ট চায়। **T**: সার্ভার ও বায়োমেট্রিক যন্ত্র কিনতে হবে। **E**: ছয়জন বছর সাশ্রয়, লাইসেন্স খরচ তিন, পেব্যাক পাঁচ বছর? **L**: রোগীর তথ্য দেশের ভেতরেই থাকতে হবে। **O**: কাউন্টারের কর্মী সব তথ্য আবার টাইপ করবেন না। **S**: কোয়ার্টার শেষের আগে চালু? কেবল T, L ও O পাশ — প্রকল্প অপেক্ষায়।

৬ অ্যানালাইসিস মডেল ও ডেটা ফ্লো ডায়াগ্রাম

অ্যানালাইসিস মডেল: প্রয়োজনীয়তা থেকে তৈরি সিস্টেমের **বিমূর্ত প্রতিনিধিত্ব**। **DFD = কী ঘটে** (ফাংশন ও প্রবাহ) · **ERD = কী আছে** (তথ্যের কাঠামো) — দুটো পরিপূরক, প্রতিদ্বন্দ্বী নয়।

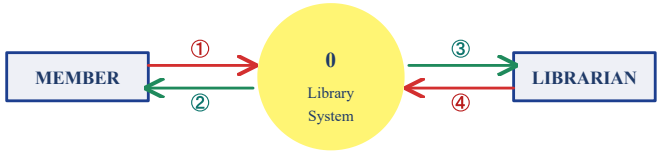
উপাদান	চিহ্ন	অর্থ
বাইরের এন্টিটি	বর্গাকার	সিস্টেমের বাইরের উৎস বা গন্তব্য
প্রসেস	বৃত্ত	ইনপুট ডেটাকে আউটপুটে রূপান্তর; 1.0, 2.0 …
ডেটা স্টোর	দুই রেখা	সিস্টেমের ভেতরে সংরক্ষিত সম্পর্কিত তথ্য
ডেটা ফ্লো	তির	উপাদানগুলোর মধ্যে চলা একটি **নাম দেওয়া ডেটা**



ব্যালান্সিং: নিচের স্তর পিতা প্রসেসের প্রবাহ না বদলে বদলায়

৭ লেভেল ০: কনটেক্সট ডায়াগ্রাম

লেভেল ০ = কনটেক্সট ডায়াগ্রাম ★: **পুরো সিস্টেমকে একটি মাত্র প্রসেস** হিসেবে দেখানো; সিস্টেম সীমানা পেরিয়ে যাওয়া প্রবাহ দিয়ে বাইরের এন্টিটির সঙ্গে যুক্ত।
কোনো স্টোর বা অভ্যন্তরীণ প্রসেস থাকে না — সেগুলো লেভেল ১-এ আসে।



① ধার বা সার্চের অনুরোধ ② বই ও ডিউ-ডেট
③ ধার ও জরিমানার রিপোর্ট ④ নতুন বইয়রেকর্ড

বেশি আসে: একটি বৃত্ত = পুরো সিস্টেম

পরীক্ষার ফাঁদ: এখানে ডেটাবেজের টেবিল (সেটা ERD), অভ্যন্তরীণ প্রসেস (লেভেল ১) বা সময়ক্রম অনুযায়ী বার্তা (sequence diagram) দেখানো হয় না।

স্তর ও বিভাজন: কনটেক্সট (০) → লেভেল ১ = প্রতিটি প্রধান ফাংশনে একটি বৃত্ত → লেভেল ২, ৩ … নম্বরকরণ: লেভেল ১-এ 1.0, 2.0; লেভেল ২-এ 2.1, 2.2। জটিলতা সামলাতে **divide and conquer**।

নিয়ম (DFD): স্টোর → স্টোর নয়, স্টোর → এন্টিটি নয়, এন্টিটি → স্টোর নয়। প্রতিটি প্রসেসে ≥ ১টি ইনপুট ও ≥ ১টি আউটপুট লাগে (ধূসর বা কালো গর্ত = ভুল)। প্রবাহ **যুক্তিগত** এবং **নামযুক্ত**।

৮ ডেটা ডিকশনারি

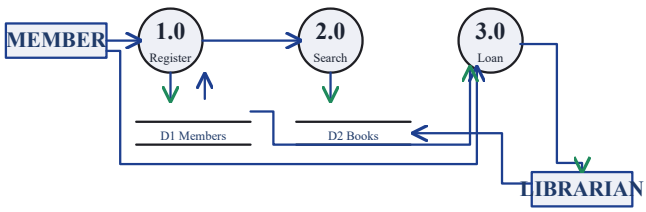
সংজ্ঞা: DFD-র প্রতিটি ডেটা ফ্লো, ডেটা স্টোর ও ডেটা উপাদানের **বিস্তারিত বর্ণনা**। প্রতিটি এন্ট্রির চারটি অংশ: **নাম = গঠন / সংজ্ঞা / কোথায় ব্যবহৃত / মন্তব্য**। যে এন্ট্রি লেখা যায় না, প্রয়োজনীয়তা এখনো ঝাপসা।

চিহ্ন	অর্থ
x = a + b	x গঠিত a ও b দিয়ে
x = a, b	বিকল্প — a **অথবা** b
x = [a]	পুনরাবৃত্তি — **এক বা তার বেশি**
x = {a}	ঐচ্ছিক — **শূন্য বা এক**
qualifier/x	x হলো সেই key, যা একটি y চিহ্নিত করে
* লেখা *	মন্তব্য / ছদ্মনাম

কেস A-এর এন্ট্রি ✍️ :

```
Loan = loan_id + member_id +
    accession_no + issue_date +
    due_date + {return_date}
Member = member_id + name +
    phone + membership_type
Book = accession_no + isbn + title
```

কেস A — একই লাইব্রেরি সিস্টেম, DFD হিসেবে



লেভেল ১: তিনটি প্রসেস, দুটি স্টোর, দুটি বাইরের এন্টিটি

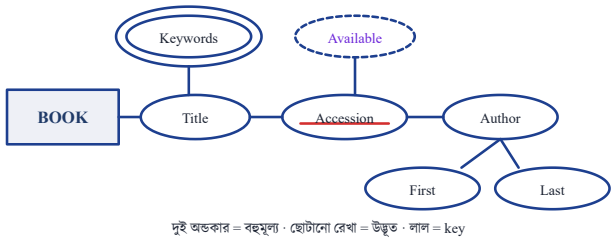
- **1.0 Register**: সদস্যের তথ্য → D1 Members।
- **2.0 Search**: ক্যাটালগ কোয়েরি → বইয়ের তালিকা; D2 পড়ে।
- **3.0 Loan**: ধার/ফেরত → D2 দেখে, D1-এ লেখে, বই ও ডিউ-ডেট ফেরত দেয়।
- **লাইব্রেরিয়ান**: D2 রক্ষণাবেক্ষণ করে, ওভারডিউ রিপোর্ট পায়।

পৃষ্ঠা ৪-এর কনটেক্সট ডায়াগ্রামের সঙ্গে ব্যালান্সড: সেখানে ঢোকা প্রতিটি প্রবাহ এখানে কোনো প্রসেস নেয়, আর বেরোনো প্রতিটি প্রবাহ এখানে তৈরি হয়।

পরীক্ষার ফাঁদ: DFD **যুক্তিগত** প্রবাহ দেখায়, তাই "book file" মানে "C:\library\books.txt" নয়। ডেটাবেজ ডিজাইন চাইলে ERD আঁকো।

৯ এন্টিটি-রিলেশনশিপ (ER) মডেলিং

সংজ্ঞা: **ER মডেল** (Chen, **১৯৭৬**) একটি **কনসেপচুয়াল তথ্য মডেল**: বাস্তব জগতকে দেখায় **এন্টিটি**, তার **অ্যাট্রিবিউট** এবং এন্টিটিগুলোর মধ্যে **সম্পর্ক** দিয়ে — কোনো DBMS বা ফাইলের উল্লেখ ছাড়াই।

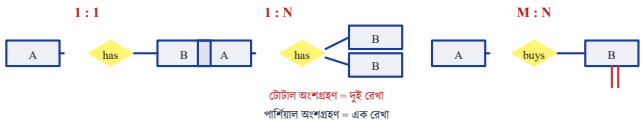


অ্যাট্রিবিউট নোটেশন: সরল, key, কম্পোজিট, বহুমূল্য, উদ্ভূত

এন্টিটি: বাস্তব জগতের বস্তু বা ঘটনা যা স্পষ্টভাবে শনাক্ত করা যায়। **শক্তিশালী (strong)** এন্টিটির নিজস্ব key থাকে; **দুর্বল (weak)** এন্টিটির (দ্বিবর্গাকার ফ্রেম, পার্টিয়াল key) নিজের key নেই, শুধু **মালিক** এন্টিটির মাধ্যমে চিহ্নিত হয়।

অ্যাট্রিবিউট: **সরল / কম্পোজিট** (নাম = প্রথম + শেষ) · **একক / বহুমূল্য** (কয়েকটি ফোন নম্বর) · **উদ্ভূত** (জন্মদিন থেকে বয়স) · **key** (নিচে রেখা, স্বল্পতম)।

সম্পর্ক (relationship): এন্টিটিগুলোর মধ্যে সংযোগ। **ডিগ্রি** = কয়টি এন্টিটি ধরন অংশ নেয়: ইউনারি (১), বাইনারি (২), টার্নারি (৩)। ইউনারিকে **রিকার্সিভ**ও বলে — কর্মী *পরিচালনা করে* কর্মী।

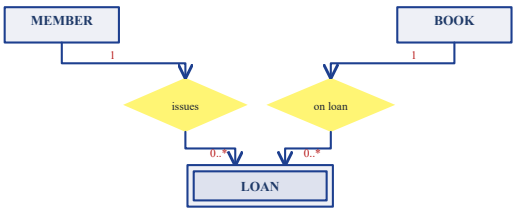


কার্ডিনালিটি (সর্বোচ্চ কত) ও অংশগ্রহণ (অবশ্যই কি?)

কার্ডিনালিটি	লাইব্রেরির উদাহরণ
1 : 1	এক সদস্য ↔ এক কার্ড
1 : N	এক সদস্য → অনেক ধার
M : N	সদস্য ↔ বই, সময়ের সঙ্গে
টোটাল	প্রতিটি উদাহরণকে **অবশ্যই** যেতে হয়
পার্শিয়াল	এন্টিটি **বাইরে থাকতে** পারে

পরীক্ষার ফাঁদ: কার্ডিনালিটি ও অংশগ্রহণ **ভিন্ন** প্রশ্ন: "সর্বোচ্চ কত?" বনাম "অবশ্যই অংশ নেবে?"। 1 : N মানে N-পক্ষ বাধ্যতামূলক নয়।

১০ কেস B — ER ডায়াগ্রাম, তারপর রিলেশনাল স্কিমা



Chen নোটেশন — LOAN দুর্বল: key = Member_id + Accession_no

একজন **MEMBER** 0..*টি **LOAN** দেন, আর একটি **BOOK** 0..*টি **LOAN**-এ থাকে; LOAN দুটির সঙ্গেই টোটাল অংশ নেয়, তাই দুটো ছাড়া তার অস্তিত্ব নেই। এতে থাকে Issue_date, Due_date ও {Return_date}। একই দুই এন্টিটি পরে আবার মেলে — ধার, অর্ডার, ভর্তি — তখন ঘটনাটি ধরে রাখতে **তৃতীয় একটি রিলেশন** লাগে।

পরীক্ষার ফাঁদ: MEMBER–LOAN এক লাইনের সাধারণ 1:N নয়: (সদস্য, বই) জোড়া সময়ের সঙ্গে আবার আসে, তাই এটি **M:N**, এবং LOAN হয় **সহায়ক (associative) এন্টিটি**।

১১ ER → রিলেশনাল ম্যাপিং: সাতটি নিয়ম

#	উৎস উপাদান	যে রিলেশন তৈরি হয়
১	শক্তিশালী এন্টিটি	একই অ্যাট্রিবিউট; তার key = **primary key**
২	দুর্বল এন্টিটি	মালিকের key আসে FK হিসেবে; PK = মালিকের key + পার্টিয়াল key
৩	1 : 1 সম্পর্ক	এক পক্ষের key **অন্য** রিলেশনে FK
৪	1 : N সম্পর্ক	**এক** পক্ষের key **অনেক** পক্ষের রিলেশনে FK
৫	M : N সম্পর্ক	**নতুন রিলেশন**: দুটি key FK; জোড়াই PK
৬	বহুমূল্য অ্যাট্রিবিউট	**নতুন রিলেশন**: এন্টিটির key + অ্যাট্রিবিউট = PK
৭	N-ary সম্পর্ক	**নতুন রিলেশন**: সব অংশগ্রহণকারীর key = PK

কেস B প্রয়োগ ✍️: *MEMBER(member_id, name, phone)*
BOOK(accession_no, isbn, title)
LOAN(member_id, accession_no, issue_date, due_date, {return_date})
AUTHOR_BOOK(isbn, author_name)
নিয়ম ২ দেয় LOAN-এর key; নিয়ম ৬ ভাগ করে বহু-লেখকের বই।

তথ্য মডেল	উত্তর দেয়
কনসেপচুয়াল	কোন এন্টিটি ও সম্পর্ক আছে — ERD
লজিক্যাল	অ্যাট্রিবিউট, ডোমেইন, key; রিলেশনাল মডেলে
ফিজিক্যাল	টেবিল, কলাম, ডেটা টাইপ, ইনডেক্স — DBMS-নির্দিষ্ট

পরীক্ষার ফাঁদ: কনসেপচুয়াল → লজিক্যাল → **ফিজিক্যাল**। কনসেপচুয়াল মডেলে **অ্যাট্রিবিউট ও key থাকে না**।

১২ ডিজাইন: প্রয়োজনীয়তা থেকে ব্লুপ্রিন্ট

নীতি	বাস্তবে
অ্যাবস্ট্রাকশন	একটি ইউনিট **কী** করে তা বলো, **কীভাবে** তা লুকাও
মডুলারিটি	আলাদা করে পরীক্ষাযোগ্য **মডিউলে** ভাগ করো
তথ্য গোপনকরণ ★	**(Parnas, 1972)** যা বদলাতে পারে তা লুকাও
উদ্বেগ পৃথকীকরণ	এক মডিউল, এক উদ্বেগ
রিফাইনমেন্ট	উপর থেকে নিচে ধাপে ধাপে কোড পর্যন্ত
পুনঃব্যবহার	বিদ্যমান ডিজাইন বা কম্পোনেন্ট ব্যবহার করো

কী লাভ: মডিউল বোঝা, একক পরীক্ষা, পুনঃব্যবহার ও স্বাধীনভাবে পরিবর্তনযোগ্য; সংশোধন এক মডিউলেই থাকে।

পরীক্ষার ফাঁদ: **অ্যাবস্ট্রাকশন ≠ এনক্যাপসুলেশন**। অ্যাবস্ট্রাকশন ঠিক করে কী দেখানো হবে; এনক্যাপসুলেশন ডেটা ও তার অপারেশন একসঙ্গে বাঁধে। **মডুলারিটি লক্ষ্য, তথ্য গোপনকরণ তার কৌশল।**

১৩ ফাংশনাল ইনডিপেন্ডেন্স: কোহিশন ও কাপলিং

কোহিশন (cohesion): একটি মডিউলের ভেতরের উপাদানগুলো একটি উদ্দেশ্যে কতটা জড়িয়ে। **যত বেশি, তত ভালো।**

#	প্রকার	জিজ্ঞাসা
১	**ফাংশনাল ★**	একটি স্পষ্ট কাজ?
২	সিকোয়েন্শিয়াল	আউটপুট → পরের ইনপুট?
৩	কমিউনিকেশনাল	একই ডেটা ভেতরে-বাইরে?
৪	প্রসিডিউরাল	নির্দিষ্ট ক্রমে?
৫	টেম্পোরাল	শুধু একসঙ্গে চলে?
৬	লজিক্যাল	যেকোনো একটি বাছা?
৭	**কয়েন্সিডেন্টাল**	শুধু একই ফাইলে?

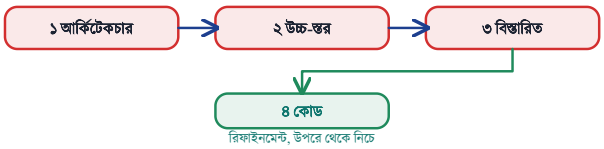
কাপলিং (coupling): একটি মডিউল অন্যটির উপর কতটা নির্ভর করে। **যত কম, তত ভালো।**

#	প্রকার	ভাগ করে
১	**ডেটা ★**	সরল প্যারামিটার
২	স্ট্যাম্প	গোটা স্ট্রাকচার
৩	কন্ট্রোল	একটি ফ্ল্যাগ
৪	এক্সটার্নাল	একটি ইন্টারফেস
৫	কমন	একটি গ্লোবাল এরিয়া
৬	**কনটেন্ট**	অন্যের **অভ্যন্তরীণ** তথ্য

COHESION ↓	COUPLING ↓
functional	**data**
sequential	stamp
communicational	control
procedural	external
temporal	common
logical	**content**
coincidental	

দুটি তালিকাই সরাসরি পরীক্ষায় আসে — এই ক্রমেই মুখস্থ করো

১৪ ডিজাইনের তিনটি স্তর



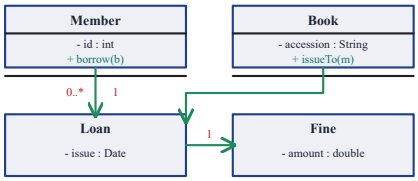
প্রতিটি স্তর উপরেরটিকে পরিশীলিত করে

স্তর	যা ঠিক করে
১ আর্কিটেকচারাল	সিস্টেমের কাঠামো: কম্পোনেন্ট, সম্পর্ক ও সীমাবদ্ধতা; তথ্য কাঠামো ও ইন্টারফেস
২ উচ্চ-স্তর (মডিউলার)	মডিউল → সাবমডিউল; কোন মডিউল কোন ডেটা স্ট্রাকচারের মালিক
৩ বিস্তারিত	সঠিক অ্যালগরিদম, রেকর্ডের নেস্টিং, ভাষাভিত্তিক ডেটা স্ট্রাকচার

১৫ আর্কিটেকচারাল ডিজাইন প্রক্রিয়া

১. ডিজাইনের ঠিকানা: ডিজাইন মডেলকে **কোন কোন ফাংশন** করতে হবে?
২. তথ্য নকশা করো — ডোমেইন মডেল থেকে **ডিজাইন ক্লাস ডায়াগ্রাম** (DCD) হয়
৩. ফাংশন বণ্টন: প্রতিটি ক্লাস কোন কম্পোনেন্ট বা সাবসিস্টেমে থাকবে
৪. **ডিজাইন ক্লাস** পরিশীলিত করো: অ্যাট্রিবিউট, অপারেশন, অ্যাসোসিয়েশন ও **নেভিগেবিলিটি**

৫ ডিজাইন সিকোয়েন্স ঠিক করো: ক্লাসগুলো **যে ক্রমে বাস্তবায়িত ও একক পরীক্ষিত হবে**



কেস C: ডোমেইন ক্লাস → ডিজাইন ক্লাস

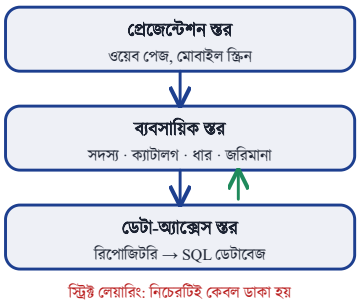
- **ডোমেইন মডেল** (শুধু তথ্য) থেকে যোগ হয় **অপারেশন**, **নেভিগেবিলিটি তির** ও প্যাকেজ।
- **কেস C**: Member 1 → 0..* Loan, Book 1 → 0..* Loan, Loan 1 → 0..1 Fine।
- ডিজাইন সিকোয়েন্স: আগে **Book** ও **Member** (এরা তথ্য ধরে), তারপর **Loan**, তারপর **Fine**।
- এই কাজের জন্যই GRASP প্যাটার্ন।

Information Expert, Creator, Low Coupling–High Cohesion

পরীক্ষার ফাঁদ: DCD ডোমেইন মডেল **নয়**: এটি বাস্তবায়ন-সচেতন, আর ডোমেইন মডেলে শুধু ক্লাস, অ্যাট্রিবিউট ও অ্যাসোসিয়েশন থাকে।

১৬ আর্কিটেকচারাল স্টাইল

স্টাইল	কম্পোনেন্ট কীভাবে কথা বলে	উদাহরণ
লেয়ার্ড	উপরের স্তরকে সেবা দেয়, নিচেরটি ব্যবহার করে	UI → ব্যবসা → ডেটা
পাইপ-অ্যান্ড-ফিল্টার	ফিল্টারের শৃঙ্খলে স্ট্রিম বয়ে যায়	Unix shell: cat f | grep x
ক্লায়েন্ট-সার্ভার	ক্লায়েন্ট সার্ভারের সেবা চায়	ব্রাউজার ও ওয়েব সার্ভার
রিপোজিটরি	সব কম্পোনেন্ট একটি স্টোর পড়ে-লেখে	যৌথ ডেটাবেজ
ইভেন্ট-চালিত	কম্পোনেন্ট ইভেন্ট ঘোষণা করে	GUI লিসেনার, কিউ
ব্রোকার	ব্রোকার অনুরোধ ও সেবা মেলায়	সার্ভিস লুকআপ, CORBA বাস
MVC	Model = তথ্য, View = রেন্ডার, Control = ইভেন্ট	ওয়েব ফ্রেমওয়ার্ক
মাইক্রোসার্ভিসেস	প্রতিটি ব্যবসায়িক সক্ষমতায় আলাদা সেবা	আলাদা পেমেন্ট সার্ভিস



লাইব্রেরি সিস্টেম তিন স্তরের আর্কিটেকচারে

লেয়ার্ড কেন জেতে ✍️: ডেটাবেজ MySQL থেকে PostgreSQL-এ গেলে কেবল ডেটা-অ্যাক্সেস স্তর বদলায়; UI বদলালে কেবল উপরের স্তর। প্রতিটি স্তর আলাদা করে পরীক্ষা করা যায়; দাম হলো প্রতিটি সীমানায় ডেটা অবজেক্টের একটি কপি।

মনে রাখার কৌশল: MVC: **M**odel = যা সত্য · **V**iew = যেভাবে দেখায় · **C**ontroller = ব্যবহারকারী এইমাত্র কী করেছে।

১৭ UI ডিজাইন: ব্যবহারযোগ্য ইন্টারফেসের নীতি

- **সিস্টেমের অবস্থা দৃশ্যমান** — কী হচ্ছে তা দেখান।
- **বাস্তব জগতের সঙ্গে মিল** — ব্যবহারকারীর ভাষা ও একক।
- **ব্যবহারকারীর নিয়ন্ত্রণ ও স্বাধীনতা** — undo, cancel, পরিষ্কার বহিরপথ।
- **ধারাবাহিকতা ও মান** — এক রূপ, এক অর্থ।
- **ভুল প্রতিরোধ** — সীমাবদ্ধ রাখো, নিশ্চিত করো, ডিফল্ট দাও।
- **স্মরণের বদলে চিনতা** — অপশন দেখিয়ে দাও।
- **অ্যাক্সেসযোগ্যতা** — প্রবীণ ও প্রতিবন্ধী ব্যবহারকারীও পারেন।

লক্ষ্য: সহজ, দক্ষ ও আনন্দময় ব্যবহার। তিনটি **স্বর্ণ নিয়ম**: **ব্যবহারকারীকে নিয়ন্ত্রণে রাখো, ব্যবহারকারীর স্মরণের বোঝা কমাও, সিস্টেমকে ধারাবাহিক রাখো**।

পরীক্ষার ফাঁদ: **ব্যবহারযোগ্যতা ≠ শেখার সহজতা**। একটি সিস্টেম শিখতে সহজ হয়েও ব্যবহারযোগ্য না হতে পারে; নতুনদের জন্য সহজ সিস্টেম দিনজুড়ে কাজ করা একজন অভিজ্ঞের জন্য অব্যবহারযোগ্যও হতে পারে।

পরীক্ষার আগের রাতের পাতা

তথ্যশালা:

অ্যানালাইসিস = **কী** · ডিজাইন = **কীভাবে**

৬ নীতি: সঠিক, ধারাবাহিক, সম্পূর্ণ

…সম্ভব, ট্রেসযোগ্য, বোধগম্য

আহরণ: ইন্টারভিউ, JAD, জরিপ, পর্যবেক্ষণ

TELOS = T, E, L, O, S

DFD: এন্টিটি, প্রসেস, স্টোর, ফ্লো

লেভেল ০ = কনটেক্সট = **১ প্রসেস**

ব্যালান্সিং: সন্তানের প্রবাহ = পিতার

স্তর: 1.0, 2.0 → 2.1, 2.2 …

কোহিশন: ফাংশনাল → কয়েন্সিডেন্টাল

কাপলিং: ডেটা → কনটেন্ট

ER: এন্টিটি, অ্যাট্রিবিউট, সম্পর্ক

ডিগ্রি = অংশগ্রহণকারীর সংখ্যা

দুই রেখা = টোটাল অংশগ্রহণ

M:N, বহুমূল্য, n-ary → নতুন রিলেশন

কনসেপচুয়াল → লজিক্যাল → ফিজিক্যাল

ডিজাইন: আর্কিটেকচারাল → উচ্চ → বিস্তারিত

লেয়ার্ড, MVC, ক্লায়েন্ট-সার্ভার

1. কনটেক্সট = একটি প্রসেস
2. স্টোর → স্টোর কখনো নয়
3. প্রতিটি প্রসেসে ইন ও আউট লাগে
4. ফাংশনাল কোহিশন সেরা
5. ডেটা কাপলিং সবচেয়ে শিথিল
6. কনটেন্ট কাপলিং সবচেয়ে খারাপ
7. M:N → জাংশন রিলেশন
8. দুর্বল key = মালিকের key + পার্টিয়াল
9. দুই রেখা = টোটাল অংশগ্রহণ
10. কনসেপচুয়াল মডেলে অ্যাট্রিবিউট নেই
11. তথ্য গোপনকরণ: Parnas, 1972
12. মাইক্রোসার্ভিস আলাদা ডিপ্লয় হয়

সবচেয়ে বড় ফাঁদ: "কী" ও "কীভাবে" অ্যানালাইসিস-ডিজাইনের মধ্যে গুলিয়ে যাওয়া · স্টোরসহ আঁকা কনটেক্সট ডায়াগ্রাম · প্রবাহকে ফিজিক্যাল স্টোরেজ ভাবা · কোহিশন ও কাপলিংয়ের ক্রম উলটো করা · M:N-কে দুটি কলামে রেখে দেওয়া · টোটাল অংশগ্রহণকে কার্ডিনালিটি ভাবা · কনসেপচুয়াল মডেলে অ্যাট্রিবিউট ঢোকানো

সব কৌশল একসঙ্গে: কনটেক্সট = একটি বৃত্ত · ইন-ফ্লোর বিরুদ্ধে আউট-ফ্লো · প্রতিটি প্রসেস নম্বরযুক্ত · প্রবাহের নাম থাকতে হবে · দুর্বল = দ্বিবর্গাকার ফ্রেম + ছোটানো রোম্বাস · নিচে রেখা = key, দুই অন্তকার = বহুমূল্য, ছোটানো = উদ্ভূত · স্ট্রিক্ট লেয়ারিং শুধু নিচে ডাকে · নিয়ন্ত্রণ, স্মরণ, ধারাবাহিকতা



প্রয়োজনীয়তা → মডেল → ডিজাইন, আবার ফিরে

আরও অনুশীলন (লিখিত): ① হাসপাতালের কনটেক্সট ও লেভেল-১ DFD, ব্যালান্সিং সহ · ② ছয়টি প্রবাহের ডেটা ডিকশনারি · ③ M:N অ্যাকাউন্ট থাকা ব্যাংকের ER → রিলেশন

পাশের লেসনে কী আছে: ইউজ কেস, ইউনিফাইড প্রসেস ও UML — ইউজ কেস / OO ডিজাইন নোটে; GRASP প্যাটার্ন — GRASP নোটে।

✓ অনুশীলনী MCQ (পরীক্ষার ধরনে, ১০ নম্বর)

১. সবচেয়ে শিথিল কাপলিং কোনটি?
ক) ডেটা কাপলিং খ) কনটেন্ট কাপলিং
গ) কমন কাপলিং ঘ) কন্ট্রোল কাপলিং

২. সবচেয়ে শক্তিশালী কোহিশন কোনটি?
ক) কয়েন্সিডেন্টাল খ) ফাংশনাল
গ) টেম্পোরাল ঘ) লজিক্যাল

৩. লেভেল ০ DFD কী দেখায়?
ক) প্রতিটি প্রসেস খ) টেবিলগুলো
গ) বার্তাসমূহ ঘ) একটি প্রসেস

৪. DFD-এ কোনটি নিষিদ্ধ?
ক) প্রসেস → স্টোর খ) প্রসেস → এন্টিটি
গ) স্টোর → স্টোর ঘ) এন্টিটি → প্রসেস

৫. দুর্বল এন্টিটির key কী?
ক) শুধু পার্টিয়াল key খ) মালিকের key + পার্টিয়াল
গ) সারোগেট key ঘ) শুধু মালিকের key

৬. M : N কীভাবে রূপান্তর হয়?
ক) নতুন রিলেশন খ) কিছু নয়
গ) নতুন কলাম ঘ) প্রসিডিউর

৭. দুই রেখা মানে কী?
ক) পার্শিয়াল অংশগ্রহণ খ) আইডেন্টিফাইং
গ) টোটাল অংশগ্রহণ ঘ) বহুমূল্য

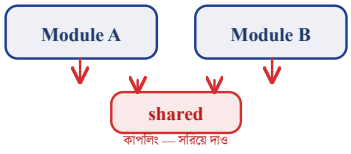
৮. বহুমূল্য কীভাবে রূপান্তর হয়?
ক) অ্যারে কলাম খ) দুটি কলাম
গ) উদ্ভূত ঘ) নতুন রিলেশন

৯. UI / ব্যবসা / ডেটা কোনটি?
ক) লেয়ার্ড খ) রিপোজিটরি
গ) ব্রোকার ঘ) পাইপ-অ্যান্ড-ফিল্টার

১০. "এটি কি বানানো যাবে?" — কোনটি?
ক) আইনগত খ) অর্থনৈতিক
গ) সময়সূচিগত ঘ) প্রযুক্তিগত

উত্তরমালা → ১-ক · ২-খ · ৩-ঘ · ৪-গ · ৫-খ · ৬-ক · ৭-গ · ৮-ঘ · ৯-ক · ১০-ঘ

লিখিত প্রশ্ন: কোহিশন ও কাপলিং কী? ভালো ডিজাইনে উচ্চ কোহিশন ও নিম্ন কাপলিং কেন থাকা উচিত? প্রত্যেকটির প্রকারভেদ ভালো থেকে খারাপ ক্রমে লেখো। ১০ নম্বর



ভেতরে উচ্চ কোহিশন, বাইরে নিম্ন কাপলিং

১ **কোহিশন** = একটি মডিউলের ভেতরের উপাদানগুলো একটি উদ্দেশ্যের জন্য কতটা জড়িয়ে; **কাপলিং** = মডিউলগুলোর পারস্পরিক নির্ভরতার মাত্রা। ৩

২ ভালো ডিজাইনে মডিউল বোঝা, পরীক্ষা, পুনঃব্যবহার ও পরিবর্তন সহজ থাকে: একটির পরিবর্তন অন্যগুলোকে সামান্যই প্রভাবিত করে। ২

৩ কোহিশন, ভালো → খারাপ: **ফাংশনাল, সিকোয়েন্শিয়াল, কমিউনিকেশনাল, প্রসিডিউরাল, টেম্পোরাল, লজিক্যাল, কয়েন্সিডেন্টাল**। ২

৪ কাপলিং, ভালো → খারাপ: **ডেটা, স্ট্যাম্প, কন্ট্রোল, এক্সটার্নাল, কমন, কনটেন্ট**। ৩

পরীক্ষার আগে যাচাই ✓

- ✓ অ্যানালাইসিসের ছয় নীতি বলতে পারি
- ✓ আহরণের চারটি পদ্ধতি বলতে পারি
- ✓ কনটেক্সট ও ব্যালান্সড লেভেল-১ DFD আঁকতে পারি
- ✓ ডেটা ডিকশনারির এন্ট্রি লিখতে পারি
- ✓ কার্ডিনালিটিসহ ER ডায়াগ্রাম আঁকতে পারি
- ✓ কোহিশন ও কাপলিংয়ের ক্রম মুখস্থ